

COMMUNICATION SYSTEMS PRACTICE

Experiment 6

Amplitude Shift Keying (ASK)

Aim

- To perform Binary Amplitude Shift Keying (BASK) on a given binary message sequence, verify correct recovery of decoded bits at the receiver, and plot the message bit sequence, carrier signal, modulated signal, demodulated (envelope) signal, and decoded bit sequence.
- To extend the above to M-ary ASK for $M = 4$ and $M = 8$, and observe the effect of increasing the modulation order on the transmitted waveform and symbol levels.

Theory

Amplitude Shift Keying (ASK) is a form of digital modulation in which the amplitude of a sinusoidal carrier is varied in discrete steps according to the input data, while the frequency and phase remain constant. The general expression for an M-ary ASK signal is:

$$s(t) = A_i \cos(2\pi f_c t), \quad 0 \leq t \leq T_s \quad (1)$$

where $A_i \in \{1, 2, \dots, M\}$ is the amplitude level corresponding to the i -th symbol, f_c is the carrier frequency, and T_s is the symbol duration.

Binary ASK (BASK / OOK) is the special case $M = 2$, where the carrier is either present (bit 1) or scaled to a lower amplitude (bit 0). For On-Off Keying (OOK), $A_0 = 0$ and $A_1 = 1$.

M-ary ASK groups $k = \log_2 M$ bits into one symbol and maps each group to one of M amplitude levels, increasing spectral efficiency at the cost of reduced noise immunity.

Transmitter: The binary bit stream is grouped into k -bit symbols. Each symbol is converted to a decimal value (0 to $M - 1$) and an amplitude level $A_i = \text{decimal value} + 1$ is assigned. The baseband pulse train is then multiplied by the carrier $\cos(2\pi f_c t)$ to produce the ASK signal.

Receiver (Non-coherent envelope detection):

1. **Rectification:** The received ASK signal is full-wave rectified to obtain $|s(t)|$.
2. **Low-Pass Filtering (LPF):** A Butterworth LPF with cutoff $f_c/2$ removes the double-frequency component and retains the envelope.
3. **Decision:** The mean of each symbol interval of the filtered envelope is computed and compared against M uniformly spaced thresholds to recover the transmitted symbol. The symbol is then converted back to bits.

The relevant parameters for this experiment are summarised in Table 1.

Code

Part 1: Binary ASK ($M = 2$)

```

1  fc = 5; fs = 100; Tb = 1;
2  bits = [1 1 0 0 1 0 0 1];
3  M = 2; k = log2(M);
4
5  % Transmitter
6  symbols = reshape(bits, k, [])';
7  amp = bi2de(symbols, 'left-msb') + 1;
8  msg = repelem(amp, fs*Tb);
9  t = (0:length(msg)-1)/fs;
10 ask = msg .* cos(2*pi*fc*t);
11
12 % Receiver: rectify -> LPF -> decide
13 envelope = abs(ask);
14 [b, a] = butter(5, (fc/2)/(fs/2));
15 envelope = filter(b, a, envelope);
16 sps = fs*Tb; n = length(amp);
17 dec = zeros(1,n);
18 for i = 1:n
19     dec(i) = mean(envelope((i-1)*sps+1 : i*sps));
20 end
21 dec = max(1, min(M, round(dec * M/max(dec))));
22 dec_bits = de2bi(dec-1, k, 'left-msb'); dec_bits = dec_bits
    (:)' ;
23
24 % Plot
25 figure

```

```

26 subplot(5,1,1); stairs(t, msg); title('Message');
    grid on;
27 subplot(5,1,2); plot(t, cos(2*pi*fc*t)); title('Carrier');
    grid on;
28 subplot(5,1,3); plot(t, ask); title('ASK M=2');
    grid on;
29 subplot(5,1,4); plot(t, envelope); title('Envelope (
    LPF)'); grid on;
30 subplot(5,1,5); stairs(t, repelem(dec,sps)); title('Decoded')
    ; grid on;
31 fprintf('Original: %s\nDecoded: %s\n', num2str(bits),
    num2str(dec_bits));

```

Part 2: 4-ary ASK ($M = 4$)

```

1 fc = 5; fs = 100; Tb = 1;
2 bits = [1 1 0 0 1 0 0 1];
3 M = 4; k = log2(M);
4
5 % Transmitter
6 symbols = reshape(bits, k, [])';
7 amp = bi2de(symbols, 'left-msb') + 1;
8 msg = repelem(amp, fs*Tb);
9 t = (0:length(msg)-1)/fs;
10 ask = msg .* cos(2*pi*fc*t);
11
12 % Receiver
13 envelope = abs(ask);
14 [b, a] = butter(5, (fc/2)/(fs/2));
15 envelope = filter(b, a, envelope);
16 sps = fs*Tb; n = length(amp);
17 dec = zeros(1,n);
18 for i = 1:n
19     dec(i) = mean(envelope((i-1)*sps+1 : i*sps));
20 end
21 dec = max(1, min(M, round(dec * M/max(dec))));
22 dec_bits = de2bi(dec-1, k, 'left-msb'); dec_bits = dec_bits
    (:)' ;

```

```

23
24 % Plot
25 figure
26 subplot(5,1,1); stairs(t, msg); title('Message')
    ; grid on;
27 subplot(5,1,2); plot(t, cos(2*pi*fc*t)); title('Carrier')
    ; grid on;
28 subplot(5,1,3); plot(t, ask); title('ASK M=4')
    ; grid on;
29 subplot(5,1,4); plot(t, envelope); title('Envelope
    (LPF)'); grid on;
30 subplot(5,1,5); stairs(t, repelem(dec,sps)); title('Decoded')
    ; grid on;
31 fprintf('Original: %s\nDecoded: %s\n', num2str(bits),
    num2str(dec_bits));

```

Part 3: 8-ary ASK ($M = 8$)

```

1 fc = 5; fs = 100; Tb = 1;
2 % 8-ASK needs k=3 bits/symbol; use a 24-bit sequence
3 bits = [1 1 0 0 1 0 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 1 1 0];
4 M = 8; k = log2(M);
5
6 % Transmitter
7 symbols = reshape(bits, k, [])';
8 amp = bi2de(symbols, 'left-msb') + 1;
9 msg = repelem(amp, fs*Tb);
10 t = (0:length(msg)-1)/fs;
11 ask = msg .* cos(2*pi*fc*t);
12
13 % Receiver
14 envelope = abs(ask);
15 [b, a] = butter(5, (fc/2)/(fs/2));
16 envelope = filter(b, a, envelope);
17 sps = fs*Tb; n = length(amp);
18 dec = zeros(1,n);
19 for i = 1:n
20     dec(i) = mean(envelope((i-1)*sps+1 : i*sps));

```



```

21 end
22 dec = max(1, min(M, round(dec * M/max(dec))));
23 dec_bits = de2bi(dec-1, k, 'left-msb'); dec_bits = dec_bits
    (:)' ;
24
25 % Plot
26 figure
27 subplot(5,1,1); stairs(t, msg); title('Message')
    ; grid on;
28 subplot(5,1,2); plot(t, cos(2*pi*fc*t)); title('Carrier')
    ; grid on;
29 subplot(5,1,3); plot(t, ask); title('ASK M=8')
    ; grid on;
30 subplot(5,1,4); plot(t, envelope); title('Envelope
    (LPF)'); grid on;
31 subplot(5,1,5); stairs(t, repelem(dec,sps)); title('Decoded')
    ; grid on;
32 fprintf('Original: %s\nDecoded:  %s\n', num2str(bits),
    num2str(dec_bits));

```

Results

Table 1 lists the simulation parameters common to all three parts.

Table 1: Simulation Parameters for ASK Experiment

Parameter	Symbol	Value
Carrier Frequency	f_c	5 Hz
Sampling Frequency	f_s	100 Hz
Bit Duration	T_b	1 s
Modulation Orders	M	2, 4, 8
Bits per Symbol ($M = 2$)	k	1
Bits per Symbol ($M = 4$)	k	2
Bits per Symbol ($M = 8$)	k	3
LPF Type	—	5th-order Butterworth
LPF Cutoff	f_{cut}	$f_c/2 = 2.5$ Hz

Figures 3 shows the five-panel plots (message, carrier, ASK signal, envelope after LPF, and decoded symbols) for $M = 2$, $M = 4$, and $M = 8$ respectively.

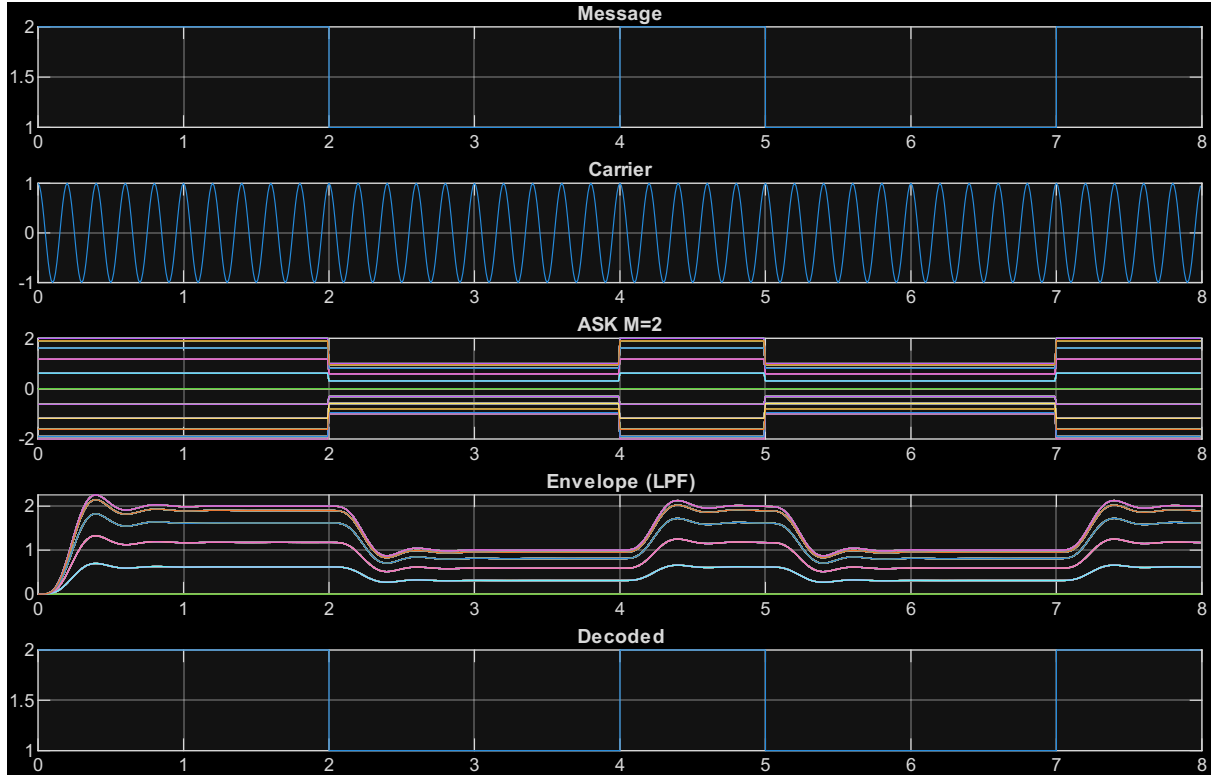


Figure 1: Binary ASK ($M = 2$): message signal, carrier signal, ASK modulated signal, envelope detected signal after LPF, and decoded symbol sequence.

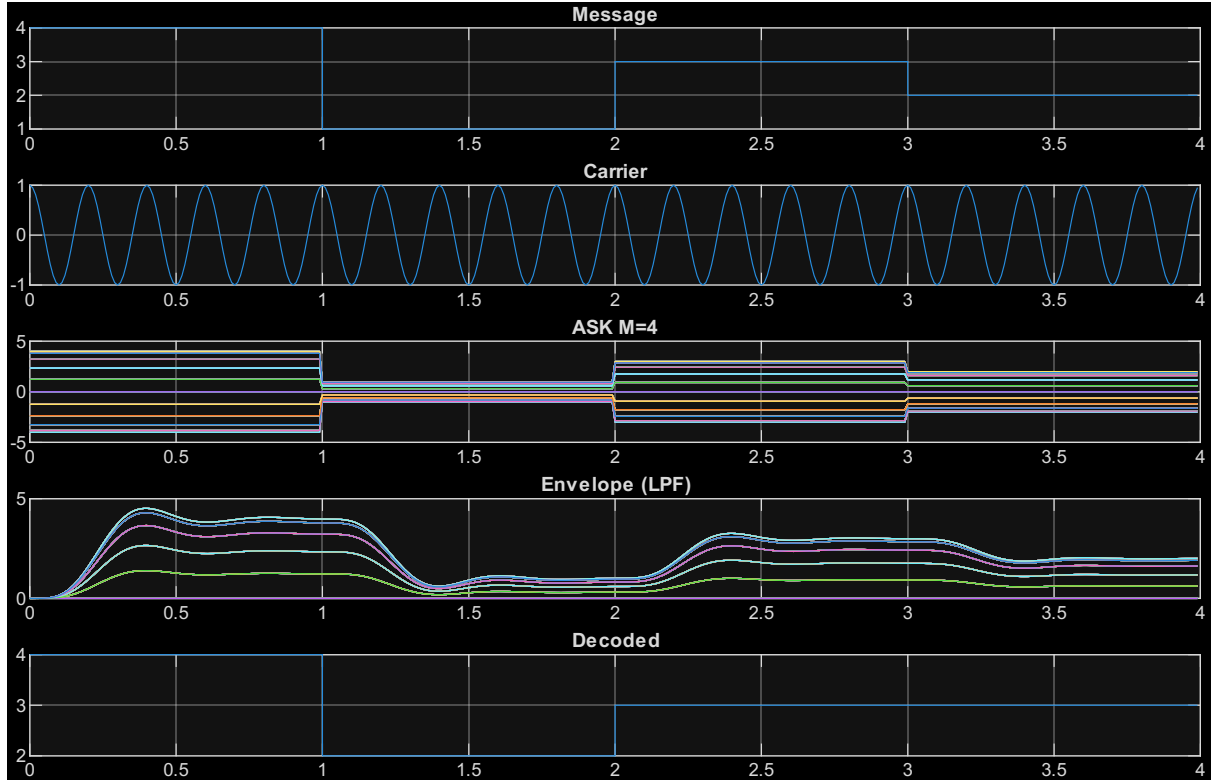


Figure 2: Binary ASK ($M = 4$): message signal, carrier signal, ASK modulated signal, envelope detected signal after LPF, and decoded symbol sequence.

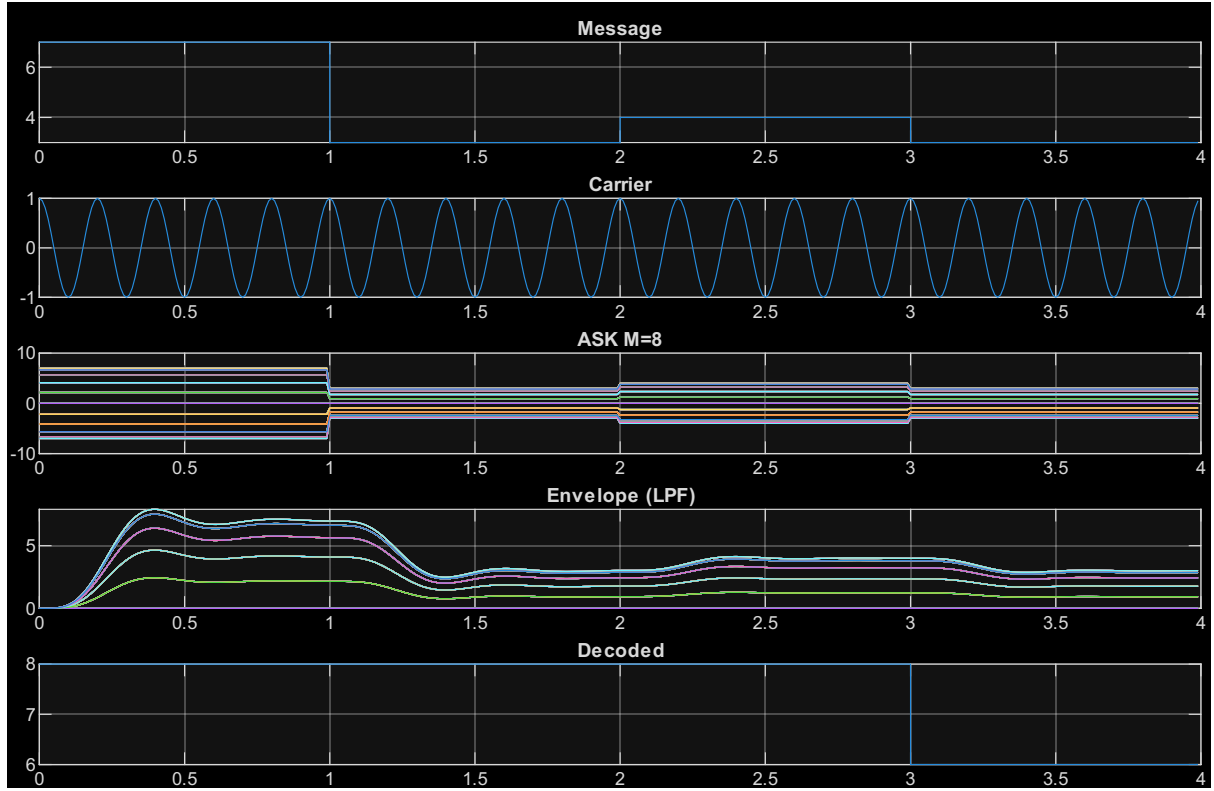


Figure 3: Binary ASK ($M = 8$): message signal, carrier signal, ASK modulated signal, envelope detected signal after LPF, and decoded symbol sequence.

Inference

Binary ASK successfully modulated the 8-bit input sequence onto a 5 Hz carrier using two amplitude levels, and envelope detection with a 5th-order Butterworth LPF recovered all bits without error. For 4-ary ASK, grouping bits into 2-bit symbols halved the number of time slots required, and four distinct amplitude levels were clearly visible in the modulated waveform. For 8-ary ASK, each 3-bit symbol occupied one slot and eight amplitude levels were observed. As M increased, the amplitude spacing between adjacent levels decreased, making the system more susceptible to noise, though all symbols were decoded correctly under noiseless simulation conditions.